MODULE 20 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

The exception hierarchy

The family tree of built-in exceptions, what abstract exceptions are, and which operation raises which error.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 20 of 28

admin@sthithapragna.com

Exceptions form a **family tree**. Every exception is also an instance of its parents, so catching a parent **catches every child** below it.

BaseException, and its three children you must know

```
1  BaseException
2   +-- SystemExit          raised by exit() or sys.exit()
3   +-- KeyboardInterrupt   the user presses Ctrl+C
4   +-- Exception           everything a program normally handles
```

1 BaseException is the root. Every exception, built-in or not, descends from it.

2-3 These two are not below Exception, so handling Exception does not stop them.

4 All the everyday errors live under Exception.

BaseException also has GeneratorExit and, since 3.11, BaseExceptionGroup. Both are beyond the syllabus.

The part of the tree the exam asks about

```
Exception
 +-- ArithmeticError
 |    +-- ZeroDivisionError
 |    +-- OverflowError
 +-- LookupError
 |    +-- IndexError
 |    +-- KeyError
 +-- TypeError
 +-- ValueError
 +-- NameError
 |    +-- UnboundLocalError
 +-- AttributeError
 +-- SyntaxError
      +-- IndentationError
```

2-4 Arithmetic problems: dividing by zero, or a result too large to represent.

5-7 Lookup problems: a bad index, or a missing key.

issubclass() answers is-a-kind-of questions

```
1  print(issubclass(ZeroDivisionError, ArithmeticError))   # True
2  print(issubclass(KeyError, LookupError))                # True
3  print(issubclass(KeyError, IndexError))                 # False: siblings
4  print(issubclass(KeyboardInterrupt, Exception))         # False
5  print(issubclass(Exception, BaseException))             # True
```

3 KeyError and IndexError share a parent, but neither is a kind of the other.

4 KeyboardInterrupt is outside the Exception branch.

Parents that group their children

EXCEPTION	GROUPS	RAISED DIRECTLY BY BUILT-INS?
`BaseException`	every exception there is	no
`Exception`	every everyday error	no
`ArithmeticError`	ZeroDivisionError, OverflowError	no: you meet its children
`LookupError`	IndexError, KeyError	no: you meet its children

ON THE EXAM

An abstract exception is one you handle, as a group, rather than one you expect to see raised.

Arithmetic and lookup errors

CODE	RAISES	PARENT
`10 / 0 10 // 0 10 % 0`	ZeroDivisionError	ArithmeticError
`2.0 ** 10000`	OverflowError	ArithmeticError
`[1][5] "ab"[9] (1,)[2]`	IndexError	LookupError
`{"a": 1}["b"]`	KeyError	LookupError

ON THE EXAM

Too large to represent is OverflowError. Very large ints are fine: only floats overflow.

Type, value, name and attribute errors

CODE	RAISES	BECAUSE
`"1" + 1 len(5) int(None)`	TypeError	the wrong type for the operation
`int("x") [1].remove(9) math.sqrt(-1)`	ValueError	the right type, but an unusable value
`print(undefined)`	NameError	no such name anywhere
`"abc".foo()`	AttributeError	the object has no such method

THE TIEBREAKER

Wrong type: TypeError. Right type, wrong value: ValueError. The most useful tiebreaker in the block.

The same function, two different errors

WRONG TYPE

TypeError

```
1  print(int(None))
```

int() cannot convert None at all: it is not a string or a number.

RIGHT TYPE, WRONG VALUE

ValueError

```
1  print(int("x"))
```

int() accepts strings, but "x" does not spell a whole number.

ON THE EXAM

Ask: could any value of this type have worked? Yes: ValueError. No: TypeError.

Other exceptions you have met

EXCEPTION	RAISED WHEN	WHERE IT SITS
`RecursionError`	recursion goes too deep	under RuntimeError
`StopIteration`	next() on a used-up generator	under Exception
`EOFError`	input() finds no more input	under Exception
`AssertionError`	an assert condition is false	under Exception
`IndentationError, TabError`	bad indentation	under SyntaxError

assert raises AssertionError when a condition is false

```
assert 2 + 2 == 4                  # true: nothing happens
assert len([]) > 0, "list is empty"
# AssertionError: list is empty
```

1. assert checks a condition that should always be true.
2. If it is false, AssertionError is raised, with the optional message after the comma.

The last line of a traceback names the exception

- A traceback starts **Traceback (most recent call last):** and lists the calls that led to the error.
- The **last line** gives the exception name, a colon, and a message.
- Since 3.11, markers under the code point to the exact part of the line that failed.
- Since 3.10, NameError and AttributeError often add: Did you mean …?

> ON THE EXAM
>
> Exam options name exceptions. Match the name; treat the message as a clue, not the answer.

What to carry into the exam

- **BaseException** is the root. SystemExit and KeyboardInterrupt sit **beside** Exception.
- ArithmeticError: ZeroDivisionError, OverflowError. LookupError: IndexError, KeyError.
- Abstract exceptions group their children; built-ins raise the children.
- Wrong type: **TypeError**. Right type, bad value: **ValueError**.
- UnboundLocalError is a NameError; IndentationError is a SyntaxError.

> Consider Python's built-in exception hierarchy.

Which exception is the direct parent of KeyError?

A `Exception`

B `ArithmeticError`

C `LookupError`

D `IndexError`

◆ PAUSE AND DECIDE

Which exception is the direct parent of KeyError?

A `Exception`

An ancestor, but not the direct parent. LookupError sits between them.

B `ArithmeticError`

Groups calculation problems, not lookups.

C `LookupError`

KeyError and IndexError are both kinds of LookupError.

D `IndexError`

IndexError is a sibling of KeyError, not its parent.

THE SIGNAL

Direct parent means one level up. KeyError, then LookupError, then Exception, then BaseException.

> Each expression is evaluated on its own.

Which two raise an exception that is a subclass of ArithmeticError? (Choose two.)

A `7 // 0`

B `int("7.0")`

C `2.0 ** 5000`

D `[7][1]`

◆ PAUSE AND DECIDE

Which two raise an exception that is a subclass of ArithmeticError? (Choose two.)

A `7 // 0`

ZeroDivisionError, a kind of ArithmeticError.

B `int("7.0")`

ValueError: a str that does not spell a whole number.

C `2.0 ** 5000`

OverflowError: the float result is too large. A kind of ArithmeticError.

D `[7][1]`

IndexError, a kind of LookupError.

THE SIGNAL

Name the exception first, then place it in the tree.

Which exception does this program raise?

```
1  data = {"a": [1, 2]}
2  print(data["a"][2])
```

A KeyError

B TypeError

C ValueError

D IndexError

◆ PAUSE AND DECIDE

Which exception does this program raise?

```
1  data = {"a": [1, 2]}
2  print(data["a"][2])
```

data["a"] succeeds: it is [1, 2]. Then [2] is past the end of that list.

A `KeyError`
The key "a" exists. The failure is in the second lookup.

B `TypeError`
A list can be indexed by an int, so the types are fine.

C `ValueError`
Bad positions are IndexError, not ValueError.

D `IndexError`
The list [1, 2] has indexes 0 and 1 only.

THE SIGNAL

With chained lookups, find which step fails, and what kind of object it was working on.

> Consider Python's built-in exception hierarchy.

Which statement is true?

A KeyboardInterrupt is a subclass of Exception

B Every exception class derives from BaseException

C A bad list index raises LookupError itself

D ValueError is a subclass of TypeError

◆ PAUSE AND DECIDE

Which statement is true?

A KeyboardInterrupt is a subclass of Exception
It sits beside Exception, directly under BaseException.

B Every exception class derives from BaseException
BaseException is the root of the whole tree.

C A bad list index raises LookupError itself
It raises IndexError, a child. LookupError is abstract.

D ValueError is a subclass of TypeError
They are siblings, both directly under Exception.

THE SIGNAL

BaseException is the root; Exception is one branch of it; abstract parents are handled, not raised.